

Albertitos · plan de arquitectura y decisiones

HackSpain 2026 · reto Maisa "500 Sombras de Alberto" · equipo: Javier Saguar, Miguel Vera, Alejandro Cuevas, Alfonso Jimena, Mónica Fernández

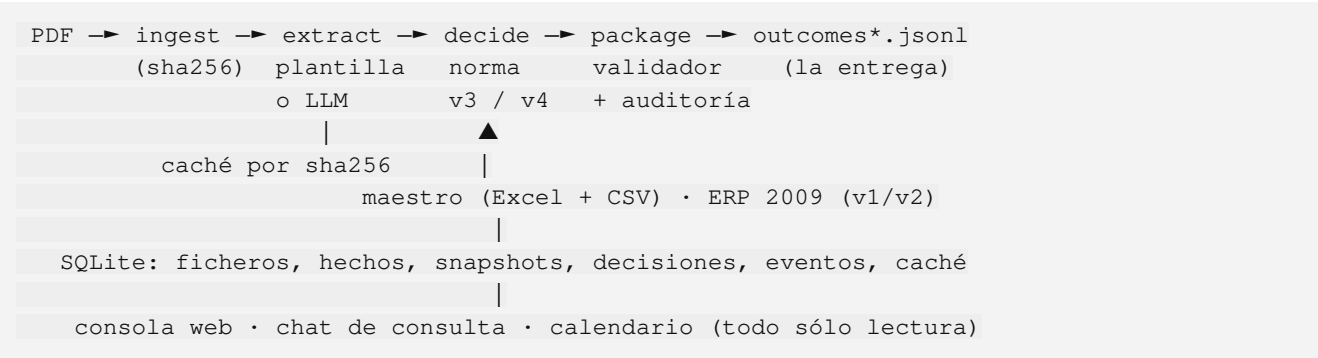
Entregado: 540 facturas decididas, **468 PAGAR · 62 ESCALAR · 10 NO_PAGAR**. Lote 1 (500) con la norma v3 y el ERP v1; lote 2 (40) con la norma v4 y el ERP v2. Auditoría en verde y 0 pendientes.

1. Arquitectura

El problema y para quién

Alberto, administrativo de cuentas a pagar del Banco Miralmar, recibe facturas PDF (algunas escaneadas), un Excel de proveedores y pedidos con su norma de pagos, y un ERP de 2009 con los asientos contables. Tiene que decidir para cada factura PAGAR, NO_PAGAR o ESCALAR, sin pagar dos veces y sin pagar lo que un humano debe ver. El sábado a las 18:00 le llegaron 40 facturas más, cuatro proveedores extranjeros en CSV y un ERP actualizado.

Componentes



- **Motor por lotes** (CLI `albertitos`): `ingest → extract → decide → package`, con eventos en cada etapa.
- **Fuentes** (`sources/`): cliente del ERP 2009 (login, token, reintento de `ORA-00600`, snapshot versionado) y loader del Excel **más los CSV del lote 2**, en un maestro versionado por el hash de su contenido.
- **Extracción** (`extract/`): `PyMuPDF` → 6 plantillas deterministas; lo que no encaja va al LLM con esquema cerrado, caché por sha256 y validadores; las escaneadas se leen dos veces y se desempatan (ADR-0017).
- **Norma como código** (`rules/`): `norma_v3.py` y `norma_v4.py`, funciones puras que devuelven motivo y evidencia por regla. Ninguna mira el reloj: la `fecha_corte` es un parámetro que se guarda con la decisión.
- **Estado** (`core/`): SQLite WAL, contratos pydantic congelados y versiones (esquema, extractor, prompt, norma).
- **Producto** (`console/`, `chat/`, `bonus/`): consola web de sólo lectura sobre un puente HTTP, chat de consulta con herramientas cerradas y calendario de pagos. Ninguno escribe decisiones.

Flujo de datos y estado

Etapas	Entra	Sale (tabla)	Evento
ingest	PDF	ficheros (sha256, nombre NFC, páginas, ¿texto?)	ingest sólo si es nuevo o cambió

extract	PDF → plantilla, o texto/ imagen → LLM	hechos (InvoiceFacts + hechos_hash), cache_llm	extract ok / pendiente / retry con tokens, coste y error
validate	todos los hechos	aviso duplicado_sospechoso (se pone y se quita)	validate con los otros PDF del grupo
enrich	bridge ERP 2009, Excel y CSV del lote 2	snapshots (ERP v1/v2, maestro por hash)	enrich por consulta (ORA-00600, 429, token)
decide	hechos + maestro + ERP + fecha_corte	decisiones (historial; vigente)	decide ok (resultado, reglas KO, por qué) o skip
emit	decisiones vigentes	dist/entrega/*.jsonl, validado, todo o nada	emit por intento y lote

Reparto entre agentes, modelos y personas

- El **modelo** extrae campos y lee las escaneadas. **No decide**: su salida es un esquema cerrado sin campo de resultado.
- El **código** decide, con motivo y evidencia por regla, y guarda con qué versión de norma, maestro y ERP lo hizo.
- La **persona** (Alberto) resuelve la cola de ESCALAR con la evidencia delante; el sistema nunca corrige a mano.
- Los **agentes de código** (Claude Code) construyeron el sistema: catorce ciclos con reparto de ficheros por agente (`docs/agentes/PLAN-NN.md`), bitácora compartida y partes con evidencia. Los contratos de `core/` estaban congelados y unos hooks impedían tocarlos, usar `date.today()` o subir secretos. No forman parte del runtime.

Observabilidad y recuperación

- **Traza**: `albertitos trace <file_id>` y la consola enseñan hechos (método, tokens, latencia), versión del maestro, asiento del ERP con sus reintentos, las reglas con su evidencia y lo que se entregó. Los eventos sólo se escriben cuando el estado cambia: repetir no ensucia la traza.
- **Si el LLM cae**: la factura queda PENDIENTE (sin hechos no hay decisión, y sin decisión no hay PAGAR) y `package` se niega a entregar. La entrega anterior sigue intacta; al volver el proveedor, el mismo `run` reanuda. Demostrado sin red en 3,7 s (`scripts/demo_caos.py --sin-red`). Tras 5 fallos seguidos, el breaker abre 60 s.
- **Sin duplicados al reanudar**: la identidad es el sha256, y la caché del LLM también. Dos pasadas dan el mismo JSONL byte a byte. Un PDF idéntico con otro nombre tiene su propia línea y las dos escalan.
- **Cambios de datos o de norma**: `reprocess --impacted` recalcula sólo lo que el cambio toca (ADR-0006). Con el IBAN de un proveedor cambiado en el Excel: 50 de 500 recalculadas, 43 cambian, 1,4 s.
- **Auditoría de entrega**: antes de escribir, comprueba conjunto exacto, NFC, duplicados, evidencias vacías y repartos raros. Si sale roja, `package` se niega; aceptarla exige un motivo escrito que queda en un evento.

Escala, evolución y coste

Medido (condiciones y comandos en `docs/CIFRAS.md`): - **Pasada completa en frío**: 500 facturas en 204 s (2,5 ficheros/s) en un i5-1235U con 4 hilos. - **Con la caché llena**: 15 s (33 ficheros/s) y 0 tokens.

Reprocesar un cambio del ERP: 2 de 500 en 0,04 s. - **Lote 2 entero** (40 facturas, 23 por plantilla y 17 por LLM de texto): extracción y decisión en menos de 20 s. - **Camino determinista a 10.000 facturas**: 57 s (Ryzen 9, sin LLM).

468 de las 500 del lote 1 salen por plantilla (3 ms cada una); el LLM lee 32. **Coste marginal 0 € por**

factura (modelos abiertos en suscripción plana): el coste es el fijo, 399 €/mes, es decir 0,04 € por factura con 10.000 al mes. El cuello de botella es la visión con doble lectura: 0,065 ficheros/s con 4 hilos y 0,106 con 8. A 10.000 facturas, las ~580 escaneadas serían ~1,5 h con una sola key (extrapolado). Escalar 10x no pide más máquina ni más gasto de LLM, sino que lleguen menos facturas a él (más plantillas) o más keys en paralelo. **Un formato nuevo** (email, Excel, imagen suelta) es un conector de `extract/` que produce el mismo `InvoiceFacts`: la norma no cambia.

Lo que el lote 2 obligó a cambiar, y lo que no

No cambió nada del motor. Se añadieron: el maestro que suma los CSV nuevos (15 proveedores, 555 pedidos), el snapshot v2 del ERP (556 asientos), un campo `moneda` en los hechos, un aviso para lo escrito a mano y la norma v4. Las 40 se decidieron con `reprocess --impacted --lote 2` en 0,03 s, sin tocar las 500 del lote 1.

2. ADRs / trade-offs

Cinco decisiones; el detalle, con alternativas y evidencia, está en `docs/adr/` (23 ADRs).

ADR-0001 · El LLM extrae; la norma decide

Hay 31 PDFs que intentan dictar la decisión («escalar», «ignorar el NIF», «pagar el total impreso») y la validación es binaria. Por eso el LLM sólo rellena `InvoiceFacts`, un esquema cerrado sin campo de decisión; lo que el PDF ordena se guarda como aviso y evidencia. Decide `rules/`: funciones puras con motivo y evidencia, versionadas (la v4 del lote 2 es un módulo nuevo, la v3 no se tocó). 468 de 500 facturas salen por plantilla determinista, el LLM lee 32 y el coste marginal es 0 €. Riesgo aceptado: escalamos de más a cambio de cero PAGAR inducidos por el documento. Si el LLM cae, PENDIENTE, nunca PAGAR.

ADR-0006 · Reprocesar sólo lo que el cambio toca

Una decisión depende de sus hechos, de la norma, de la fecha de corte y del pedido y el NIF que trae la factura en el maestro y el ERP. `reprocess --impacted` recalcula sólo las decisiones cuyo pedido o NIF toca el diff de maestro o de ERP, y dice por qué lo hace. Las demás conservan la versión con la que se decidieron y dejan un evento «sin impacto»: el historial no se reescribe. Medido: un asiento que pasa a PAGADA recalcula 1 de 500 en 0,03 s; el IBAN de un proveedor, 50 de 500 con 43 cambios en 1,4 s. Riesgo aceptado: depende de que la norma lea sólo esas claves. Un test lo comprueba en cada norma registrada, y `--todo` es la salida.

ADR-0017 · Cuando dos lecturas de una escaneada no coinciden, decide una prueba independiente

En 10 de las 29 escaneadas las dos lecturas discrepaban en NIF, IBAN o pedido, y antes ganaba la que coincidía con el maestro. Eso era comprobar el maestro contra sí mismo: la regla del proveedor ya no podía fallar. Alternativa descartada: votar dígito a dígito, que en una trampa real componía el NIF bueno. Decisión: una tercera lectura y mayoría por valor entero para los identificadores; para los importes, la lectura cuyas cuentas cuadran. Lo que no se resuelve, escala. Evidencia: sobre 26 escaneadas con respuesta conocida, 26 aciertos frente a 23 de la versión anterior; siete facturas legibles dejaron de escalar por un dígito mal leído. Riesgo aceptado: dos lecturas pueden coincidir en el mismo error. Para pagar mal, tendrían que coincidir además con el IBAN del maestro sin haberlo visto.

ADR-0021 · Cada lote se decide en su contexto

El lote 2 trajo un ERP nuevo que deja pagado un pedido del lote 1, ya entregado. Aplicarlo a todo habría cambiado una factura que la organización ya tenía validada. Decisión: cada lote conserva la norma y el ERP con los que se decidió (el 1 con v3 y v1; el 2 con v4 y v2), y `reprocess` se niega si una orden cambiaría el contexto del otro lote sin decirlo. Un duplicado entre lotes marca sólo la factura del lote

posterior: la que llegó primero no se reabre. Evidencia: con el ERP v2 aplicado al lote 1 cambiaría exactamente una factura (`factura_4635`, a `NO_PAGAR`); se midió y no se aplicó. Riesgo aceptado: conviven dos normas y dos ERP en la misma base de datos. Cada decisión guarda cuáles usó, y la traza lo enseña.

ADR-0022 · Con divisas, sin tipo de cambio acordado no se convierte: se escala

Ocho facturas del lote 2 vienen en USD, GBP, CHF, BRL, MXN o JPY contra pedidos en euros, y siete declaran IVA 0 % de exportación. La Caja no trae ninguna tabla de cambios. Los tipos implícitos son fijos por moneda y, convertidos, cuadran al céntimo con el pedido: la conversión es tentadora, pero inventar un tipo es decidir con un dato que no tenemos. Decisión: la moneda es un hecho más (`InvoiceFacts.moneda`, ISO 4217) y la regla nueva escala la factura diciendo su divisa, el importe del pedido en euros y el tipo implícito. La tabla de cambios está vacía a propósito. Si la organización publica los tipos, se rellenan y la norma compara en euros con tolerancia de redondeo; el calendario de pagos, mientras tanto, aparta lo que no esté en euros en vez de sumarlo. Riesgo aceptado: escalamos 8 facturas que quizá fueran pagables. Preferimos un escalado explicable a un pago con un tipo de cambio inventado.